

- Laboratorul 10 - *Sisteme de criptare asimetrice*

Disclaimer: Pe parcursul acestui curs/laborator vi se vor prezenta diverse noțiuni de securitate informatică, cu scopul de a învăța cum să securizați sistemele. Toate noțiunile și exercițiile sunt prezentate în scop didactic, chiar dacă uneori se presupune să gândiți ca un adversar. Nu folosiți aceste tehnici în scopuri malițioase! Acestea pot avea consecințe legale în cazul comiterii unor infracțiuni, pentru care **deveniți pe deplin răspunzători!**

1. Factorizarea modului RSA



Reamintiți-vă cum funcționează *RSA*. Țineți minte că, pentru a obține aceeași securitate, dimensiunea cheii *RSA* trebuie să fie considerabil mai mare decât dimensiunea cheii *AES*.



Se consideră cheia publică *RSA* cu modulul pe 128 biți:

$N=2348411136411758273000763594354834942653$
 $e=65537$

Factorizați modulul, i.e. determinați valorile p și q [1]. Calculați apoi coeficientul de decriptare d [2].

2. Generarea cheilor *RSA* folosind *OpenSSL*



Răspundeți la următoarele cerințe:

- a) Folosind *OpenSSL* [3], generați pentru *Alice* o cheie *RSA* pe 2048 biți, stocată într-un fișier *alice_sk.pem*.
- b) Care este valoarea exponentului de criptare?
- c) Decodați această cheie. Aflați valoarea modului N și a celor două numere prime p și q .
- d) Cheia lui *Alice* nu este protejată în niciun fel, deci este vulnerabilă. Alegeți o parolă puternică și generați o nouă cheie protejată folosind această parolă și *AES256*.
- e) Ce diferențe observați? Decodați această cheie folosind parola folosită la creare.
- f) Care este valoarea exponentului de criptare? Ce observați? Impactează această alegere securitatea?

- g) Exportați cheia publică a lui *Alice* în fișierul *alice_pk.pem*. Decodați această cheie pentru a vedea valorile modulului și exponentului.

3. Criptarea RSA și criptarea hibridă



Răspundeți la următoarele cerințe:

- Jucați rolul lui *Bob*. Criptați fișierul *bob_message.txt* folosind *RSA* [4] și cheia generată anterior. Încercați să criptați fișierul *bob_message.rtf* folosind *RSA* [4] și cheia generată anterior. Ce observați? De ce se întâmplă aceasta?
- Folosiți criptarea hibridă pentru a cripta mesajul lui *Bob* către *Alice*. Pentru aceasta, generați o cheie simetrică pe 256 biți (32 bytes) și folosiți această cheie pentru criptarea fișierului *bob_message.rtf* cu *AES-CTR* [5]. Criptați noua cheie asimetric, folosind *RSA*.
- Jucați rolul lui *Alice*. Folosiți fișierele criptate primite (criptarea cheii *AES* folosind *RSA* și criptarea mesajului folosind *AES-CTR*), decriptați și obțineți mesajul inițial.

Notă: Se poate folosi direct *smime* [6] pentru criptarea hibridă, care combină în mod direct criptarea asimetrică și criptarea simetrică.

4. Noțiuni introductive – Semnături digitale



Reamintiți-vă despre *semnături* și *certificate digitale* [7,8].



Următoarele întrebări se referă la certificatul digital al site-ului facultății [9]:

- Cine a emis certificatul digital?
- Care este validitatea certificatului?
- Pe câți biți este definită cheia publică?
- Care este valoarea exponenților de criptare din certificat și din certificatele care îl atestă în lanț? Ce observați? Are aceasta impact asupra securității?

5. Generarea cheilor folosind Putty



Folosiți *Putty* [4] pentru generarea cheilor SSH:

- Descărcați *puttygen.exe* [10].

- b) Generați o pereche de chei cheie publică - cheie privată RSA pe 2048 de biți. Apăsați *Generate*.
- c) Adaugați *PassPhrase*. La ce folosește aceasta?
- d) Exportați cheia publică în fișierul *public_key.pub*, cheia secretă în fișierul *private_key.ppk*. Pentru aceasta, folosiți *Save public key*, respectiv *Save private key*.
- e) Exportați cheia în format *openssh*. Pentru aceasta, accesați *Conversions* și *Export OpenSSH key*.
- f) Deschideți și vedeți ce conțin toate fișierele generate.

6. Generarea certificatelor cu OpenSSL



Răspundeți la următoarele cerințe folosind *OpenSSL* [11]:

- d) Generați o cheie RSA. Revedeți laboratorul de criptare asimetrică dacă nu vă mai amintiți cum realizați acest lucru.
- e) Folosiți cheia generată anterior într-un certificat *self-signed*, valabil 120 de zile, stocat ca *ca.crt*. Folosiți următoarele informații:

Country: RO

Provincie: Muntenia

Localitate: Bucuresti

Numele organizatiei: CA_SSI

Numele unitatii: CA_SSI_Lab

Common Name: CA_numele vostru (e.g.: CA_Andrei)

E-mail: test@test.ro

Pentru aceasta, folosiți comanda

```
openssl req -new -x509 - days <days> -key <key> -out ca.crt
```

- f) Citiți despre standardul *X.509* [12]. Vizualizați certificatul digital creat:

```
openssl x509 -text -noout -in ca.crt
```

- g) Folosiți acest certificat al CA ca să semnați/emiteți un alt certificat al unei entități subordonate *SUB_SLA*. Folosiți următoarele informații:

Country: RO

Provincie: Muntenia

Localitate: Bucuresti

Numele organizatiei: SUB_SSI

Numele unitatii: SUB_SSI_Lab

Common Name: SUB_numele vostru (e.g.: CA_Andrei)

E-mail: test_sub@test.ro

Pentru aceasta, mai întâi generați o nouă cheie a entității SUB_SSI pe 2048 de biți în fișierul *sub.key*.

h) Inițiați un *Certificate Signing Request (CSR)* *sub.csr*:

```
openssl req -new -key sub.key -out sub.csr
```

i) Creați apoi un certificat pentru SUB_SSI *sub.crt* semnat de autoritatea CA, valabil pentru 60 de zile, cu numărul serial 02:

```
openssl x509 -req -days <days> -in sub.csr -CA <certificat_CA> -CAkey <ca_key> -set_serial <serial_no> -out sub.crt
```

j) Vizualizați certificatul digital creat:

```
openssl x509 -text -noout -in sub.crt
```

k) Transformați acest certificat digital în *PKCS#12*:

```
openssl pkcs12 -export -out sub.p12 -inkey sub.key -in sub.crt -chain -CAfile ca.crt
```

l) Verificați conținutul fișierului *sub.p12* folosind:

```
openssl pkcs12 -info -in sub.p12
```

Referințe bibliografice

1. D.Alpern. *Integer factorization calculator*. Accesibil la: <https://www.alpertron.com.ar/ECM.HTM>.
2. Wolfram Alpha Widget – Modulo. Accesibil la: <https://www.wolframalpha.com/widgets/view.jsp?id=570e7445d8bdb334c7128de82b81fc13>.
3. OpenSSL – *rsa*. Accesibil la: <https://www.openssl.org/docs/man1.1.1/man1/openssl-rsa.html>
4. OpenSSL – *pkeyutl*. Accesibil la: <https://www.openssl.org/docs/man1.1.1/man1/openssl-pkeyutl.html>
5. OpenSSL – *openssl-enc*. Accesibil la: <https://www.openssl.org/docs/man3.0/man1/openssl-enc.html>
6. OpenSSL – *smime*. Accesibil la: <https://www.openssl.org/docs/man1.1.1/man1/openssl-smime.html>
7. Kryszczuk, K., & Richiardi, J. (2014). *Springer Encyclopedia of Cryptography and Security*. Accesibil la: https://www.researchgate.net/publication/230674947_Springer_Encyclopedia_of_Cryptography_and_Security.

8. Itfreetraining. *What are certificates?* Accesibil la: https://www.youtube.com/watch?v=LRMBZhdFjDI&ab_channel=itfreetraining.
9. Facultatea de Matematică și Informatică. Universitatea din București. Accesibil la: <https://fmi.unibuc.ro/>.
10. Putty. Accesibil la: <https://www.chiark.greenend.org.uk/~sgtatham/putty/>
11. OpenSSL. Accesibil la: <https://www.openssl.org/>.
12. <https://www.ssl.com/faqs/what-is-an-x-509-certificate/>